School of Future Tech

Case Study Report

on

Library Management System Using C++

by

Yash Rajendra Tambade

150096725173

Index

1. **Introduction to the Case Study.**

   Libraries play an important role in educational institutions. Managing books manually can be time-consuming and error-prone. This case study focuses on designing and implementing a **Library Management System using C++** that automates basic library operations such as adding books, issuing books, returning books, and displaying available inventory.
   The system demonstrates the practical implementation of **Object-Oriented Programming (OOP)** concepts such as classes, objects, arrays, loops, and conditional statements. It provides a simple console-based interface for managing book records efficiently.

---

2. **Problem Statement / Case Background (Abstract).**

   ## Background
   Traditional manual library systems require physical registers to maintain book records. This method is inefficient and prone to human error. A computerized system can improve speed, accuracy, and data organization.

   ## Abstract
   This case study presents the design and implementation of a **Library Management System using C++**. The system stores book details such as ID, title, author, and issue status. It allows users to:

   - Add new books
   - Issue books
   - Return books
   - Display all books

   The program uses a class-based structure and a menu-driven approach. The system efficiently handles up to 100 books using arrays and demonstrates structured programming practices.

---

3. **Problem Statement / Case Study Design.**

   The Library Management System is designed using a modular and menu-driven structure.

   ### System Components

   1. Book Class
   Stores:

- Book ID
- Title
- Author
- Issue Status

  Functions:

- `addBook()` → Adds book details
- `display()` → Displays book information

   2. Main Menu System
   A continuous loop provides options:

1. Add Book
2. Issue Book
3. Return Book
4. Display Books
5. Exit

3. Data Storage

- Uses an array of Book objects (maximum 100 books).
- Uses a counter variable to track total books.

---

4. **Methods & Algorithms Technology Applied in the Problem Statement / Case Study.**

Programming Concepts Used

Object-Oriented Programming (OOP)

- Class: Book
- Objects: Array of Book objects
- Encapsulation of data and functions

Control Structures

- `while(true)` loop for menu repetition
- `switch` statement for handling user choices
- `if-else` conditions for issue/return validation

Searching Algorithm

- Linear Search
    - Used to find a book by ID during issue and return operations.
    - Time Complexity: O(n)

Data Handling

- String handling using `<string>`
- Input/output using `<iostream>`

Technology Stack Used

- Programming Language: C++
- Compiler: CodeBlocks / Turbo C++ / VS Code
- Libraries Used:
  - `<iostream>`
  - `<string>`
  - `<vector>` (included but not used)
- Platform: Console Application

---

5. **Problem Statement / Case Study Implementation Details and Snapshots.**

Implementation Structure

1. Book Class Definition

Contains:

- Data members: id, title, author, isIssued
- Member functions: addBook(), display()
  Code -

```
class Book {
public:
    string id;
    string title;
    string author;
    bool isIssued;

    void addBook(string bId, string bTitle, string bAuthor) {
        id = bId;
        title = bTitle;
        author = bAuthor;
        isIssued = false;
    }

    void display() {
        cout << "ID: " << id
             << " | Title: " << title
             << " | Author: " << author;
        cout << " | Status: "
             << (isIssued ? "Issued" : "Available")
             << endl;
    }
};
```

2. Add Book Operation

- Takes user input
- Stores details in array
- Sets status to Available

Code -

```
case 1: {
    string id, title, author;
    cout << "Enter Book ID: ";
    cin >> id;
    cin.ignore();

    cout << "Enter Title: ";
    getline(cin, title);

    cout << "Enter Author: ";
    getline(cin, author);

    library[bookCount].addBook(id, title, author);
    bookCount++;

    cout << "Book added successfully!" << endl;
    break;
}
```

Output -

```
--- Library Management System ---
1. Add Book
2. Issue Book
3. Return Book
4. Display Books
5. Exit
Enter choice: 1
Enter Book ID: 101
Enter Title: Adventure
Enter Author: Harold Robbins
Book added successfully!

--- Library Management System ---
1. Add Book
2. Issue Book
3. Return Book
4. Display Books
5. Exit
Enter choice:
```

3. Issue Book Operation

- Accepts Book ID
- Performs linear search
- Updates issue status

Code -

```
case 2: {
    string id;
    cout << "Enter Book ID to issue: ";
    cin >> id;

    for (int i = 0; i < bookCount; i++) {
        if (library[i].id == id) {
            if (!library[i].isIssued) {
                library[i].isIssued = true;
                cout << "Book issued successfully!" << endl;
            } else {
                cout << "Book is already issued!" << endl;
            }
            goto next;
        }
    }

    cout << "Book not found!" << endl;
next:
    break;
}
```

Output -(If already issued)

```
Enter choice: 2
Enter Book ID to issue: 101
Book issued successfully!
```

4. Return Book Operation

- Accepts Book ID
- Marks book as Available

Code -

```
case 3: {
    string id;
    cout << "Enter Book ID to return: ";
    cin >> id;

    for (int i = 0; i < bookCount; i++) {
        if (library[i].id == id) {
            library[i].isIssued = false;
            cout << "Book returned successfully!" << endl;
            goto back;
        }
    }

    cout << "Book not found!" << endl;
back:
    break;
}
```

Output -

```
Enter choice: 3
Enter Book ID to return: 101
Book returned successfully!
```

5. Display Books

6. Loops through array

7. Calls display() function for each book
   Code -

```
case 4: {
    cout << "\n--- Current Inventory ---" << endl;

    for (int i = 0; i < bookCount; i++) {
        library[i].display();
    }
    break;
}
```

Output -

```
Enter choice: 4

--- Current Inventory ---
ID: 101 | Title: Adventure | Author: Harold Robbins | Status: Available
```

---

8. **Problem Statement / Case Study Results and Conclusion.**

Results

- The system successfully performs all four operations.
- Linear search correctly finds books by ID.
- Menu-driven interface allows easy interaction.
- The program handles up to 100 books efficiently.

Limitations

- Data is not stored permanently (no file handling).
- Maximum limit of 100 books.
- Uses `goto` (not recommended in professional applications).

Conclusion

This case study demonstrates the practical application of C++ programming and object-oriented concepts in developing a basic Library Management System. The project successfully automates book management operations and provides a strong foundation for building more advanced systems with database and file storage support.

---

9. **References**

- Online C++ tutorials (Youtube, W3Schools)
- Class notes and academic material